TD 1 - Concepts de base de ThreeJS et première scène 3D

Partie 1 : Concepts de base de ThreeJS

Introduction rapide

ThreeJS est une bibliothèque / librairie Javascript permettant de mettre en œuvre rapidement et aisément une scène dans un environnement 3D ainsi que les objets composant cette scène, sans avoir à coder l'ensemble des algorithmes nécessaires à la création, l'affichage, la manipulation et l'animation d'une telle scène.

La documentation ainsi qu'un nombre important d'exemples basés sur ThreeJS sont disponibles à l'adresse suivante : https://threejs.org/manual/#en/creating-a-scene

Les TDs réalisés avec ThreeJS qui suivront feront référence à la documentation disponible via ce lien.

Scene – Un élément pour les englober tous (et dans les ténèbres les lier)

L'élément *Scene* est une classe permettant de créer un environnement 3D, accueillant les divers objets 3D de votre scène 3D qui y seront placés dans le but de les faire afficher par le moteur de Three.js.

Il existe, entre les éléments de l'univers Three.js, une relation enfant-parent dont l'élément racine est *Scene* (avec quelques exceptions cependant).

Mesh – Un objet 3D de la scène

Le second type d'élément que vous manipulerez le plus est l'objet *Mesh.*

Une instance de classe *Mesh* est un objet 3D utilisable dans votre scène Three.js.

Il est composé de deux concepts, matérialisé dans le code par 2 objets :

Geometry – Correspondant à la forme géométrique de l'objet 3D

Material – Correspondant à l'aspect visuel et les options de rendu (brillance, opacité, texture …)

Par exemple, imaginons que nous souhaitions créer un cube rouge (un *Mesh* donc) dans notre scène : Sa forme cubique sera définie dans une instance de *Geometry*, et sa couleur dans un *Material*.

Light – Gestion de l'éclairage

Une scène 3D avec un ensemble d'objets de toutes formes, matières et couleurs cela est parfait, mais comme dans le monde réel, dans l'obscurité, on ne voit rien de tout cela.

Afin d'y voir un peu plus clair, il faut pouvoir ajouter à votre scène 3D une ou plusieurs sources de lumières.

Comme *Scene* et *Mesh,* les lumières (ambiante ou directe), sont gérées au travers d'un type de classe dont le nom est tout naturellement *Light.*

Il existe une grande diversité de classes *Light*, vous en manipulerez quelques-unes un peu plus loin dans ce TD/TP.

Camera – Le point de vue

La *Camera* est un élément indispensable de toute application Three.js. Cet objet spécial de la scène permet de définir la position et l'angle d'observation de la scène.

Il existe plusieurs modes de projection et différentes valeurs paramétrables pour configurer cette *Camera*, dont les détails seront abordés un peu plus tard.

Renderer – Moteur de rendu Three.js

Reste un objet majeur à aborder, celui qui permet de générer la scène 3D grâce à l'assemblage des objets précédemment cités, l'objet *Renderer*.

Ce dernier représente le moteur 3D qui sera utilisé pour générer le rendu 3D de la *Scene*, du point de vue de la *Camera,* définie plus tôt.

Il existe plusieurs types de *Renderer*, dont certains seront évoqués plus tard.

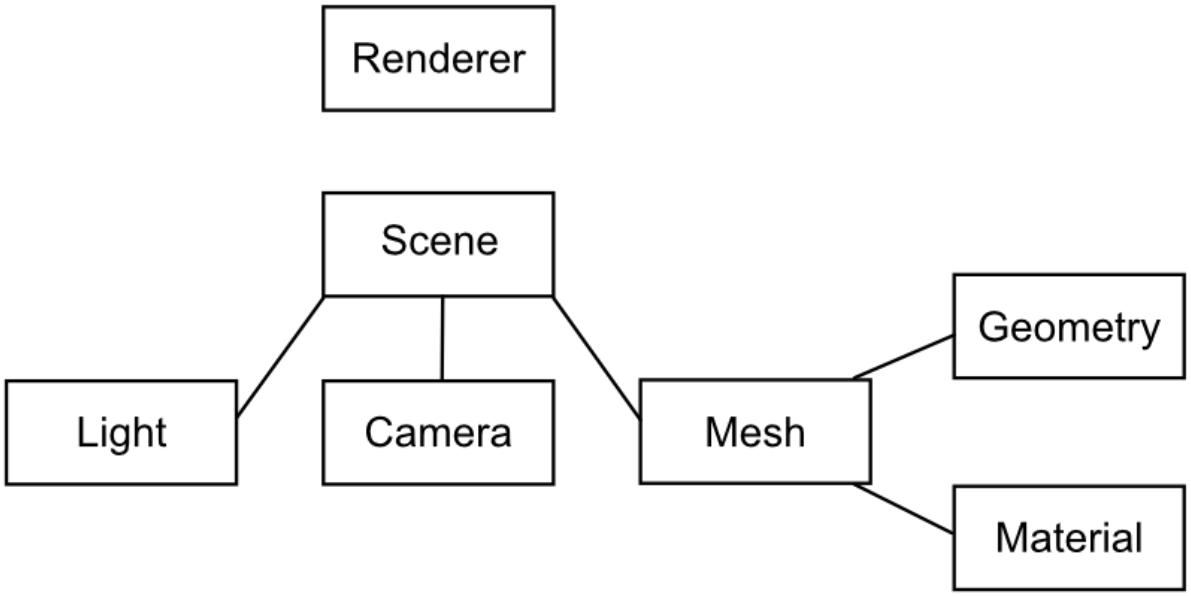


Figure 1 : Représentation du lien entre les objets composants la scène 3D dans ThreeJS

Partie 2 : Première Scène 3D

Architecture de base du projet

ThreeJS étant, comme vous l'avez vu, une bibliothèque JS, son implémentation va s'articuler autour d'une structure HTML 5, qui naturellement peut être habillée grâce au CSS.

Afin de s'assurer la meilleure portabilité et évolutivité de vos projets dans le futur en facilitant la maintenance de ces derniers, il est préférable de suivre les préconisations de la documentation officielle plutôt qu'un éventuel tutoriel en ligne.

Avec l'outil de votre choix, et dans un dossier de travail espace d'hébergement créez dès maintenant les fichiers qui contiendront votre future scène 3D.

Vous pouvez nommer vos fichiers comme ci-dessous, ou choisir vous-même les noms que vous souhaitez, à vous d'adapter le code en fonction des instructions dans ce TD guidé dans ce cas.

- index.html
- script.js
- style.css

Le contenu du fichier index.html préconisé par la documentation et qui sert de « conteneur » à notre scène, est le suivant et convient parfaitement :

Le fichier HTML, conteneur de notre scène 3D

```
<!DOCTYPE html>
<html lang="en" >
    <head>
        <meta charset="UTF-8">
        <meta name='viewport' content='width=device-width, initial-scale=1.0, maximum-scale=1.0, user-
scalable=0' />
        <title>R5.A.06 - Hello World</title>
        <link rel="stylesheet" href="./style.css">
        <script async src="https://unpkg.com/es-module-shims@2.8.2/dist/es-module-shims.js"></script>
        <script type="importmap">
          {
            "imports": {
              "three": "https://unpkg.com/three@0.185.0/build/three.module.js",
              "three/addons/": "https://unpkg.com/three@1.8.5/examples/jsm/"
            }
          }
        </script>
    </head>
    <body>
        <script type="module" src="./script.js"></script>
    </body>
</html>
```

De cette façon, le modules ThreeJS est importé, et vous allez pouvoir faire appel à ses différentes méthodes disponibles dans votre fichier script.js qui contiendra tout le code relatif à la scène 3D.

Le script JS

Vous allez maintenant vous attaquer au fichier script.js.

La première étape pour pouvoir se servir des méthodes de ThreeJS, est de récupérer une référence au module dans votre script, grâce aux imports faits dans le fichier HTML.

Pour cela, ajoutez au début de votre script :

```
import * as THREE from 'three';
```

La fonction d'initialisation de notre application

Pour cette première conception de scène 3D avec ThreeJS vous allez commencer par le sempiternel Hello World, version 3D : Un cube en 3D pivotant.

Afin de gérer plus facilement l'organisation du code, ainsi que pour contrôler plus facilement sur une scène plus complexe le rendu de la scène, vous allez encapsuler votre code dans une fonction principale d'initialisation qui sera appelée par le script.

Ajoutez une fonction *start* dans votre script *script.js :*

```
function start()
{
}
start();
```

Vous allez ensuite ajouter à votre script, et à votre fonction, les éléments constitutifs minimums de votre scène 3D via les classes disponibles dans ThreeJS, présentées précédemment et schématisées dans la *figure 1* :

- Scene
- Renderer
- Mesh
- Light
- Camera

L'élément Scene de Three.JS

Pour commencer, déclarez une variable globale pour *scene* et déclarez la dans votre fonction *start* :

```
var scene ;
function start() {
  scene = new THREE.Scene() ;
}
start() ;
```

Le constructeur de la classe Scene ne requiert ni paramètre obligatoire ni facultatif.

Il est possible d'ajouter des comportements complémentaires à la scène en définissant d'autres classes, comme notamment un brouillard, mais cela sort du cadre de ce TD.

Les Renderers de Three.js – La classe WebGLRenderer

ThreeJS fournit plusieurs Renderer, ainsi que plusieurs extensions spécifiques à ces renderers afin d'étendre leurs capacités : rendu direct dans le navigateur, pré-rendu pour un rendu différé dans le temps, rendu pour périphérique de Réalité Virtuelle.

Votre première scène (et probablement les suivantes) sera rendue directement dans le navigateur, vous allez donc utiliser la classe dédiée pour cela : WebGLRenderer

Pour cela, vous aller procéder de la même manière que pour la scène, en créant une variable globale qui sera initialisée dans votre fonction *start* :

```
var scene;
var renderer;
function start()
{
	scene = new THREE.Scene();
	// ---------------- RENDERER ----------------
	renderer = new THREE.WebGLRenderer( { antialias : true } );
	renderer.setPixelRatio( window.devicePixelRatio );
	renderer.setSize( window.innerWidth, window.innerHeight );
	document.body.appendChild( renderer.domElement );
start();
```

Quelques nouvelles méthodes, complémentaires au constructeur de la classe ont été introduites :

- ***THREEWebGLRenderer*** : le constructeur de la classe, peut prendre un ou plusieurs paramètres (antialias, powerpreference,precision), ici, antialias positionné à vrai, permet d'activer le lissage des contours d'objets pour réduire l'effet d'escalier lors du rendu.
- ***setPixelRatio*** : cette méthode permet de définir un ratio d'affichage en fonction de la dimension de l'écran. L'option choisie ici (*window.devicePixelRatio*), permet de choisir le même ratio que celui du dispositif d'affichage de la machine sur laquelle s'exécute le code.
- ***setSize :*** La méthode setSize, permet de définir les dimensions du conteneur d'affichage de la scène, qui correspondront aux dimensions de l'élément html <canvas> hébergeant la scène.

Une dernière nouveauté, la propriété ***domElement*** de l'objet *renderer* permet de récupérer l'élément html du dom correspondant à la scène que vous pouvez ainsi ajouter à votre guise dans le dom du document HTML, ici directement dans la balise *body*.

La Camera – PerspectiveCamera

Une fois encore, ThreeJS propose différents types de cameras pour les scènes 3D, chacune avec ses particularités dans la représentation de la projection spatiale (orthographique, perspective (se rapprochant le plus de ce que l'œil humain perçoit, stéréo 3D (pour un usage avec les lunettes avec filtres rouges et verts / bleus).

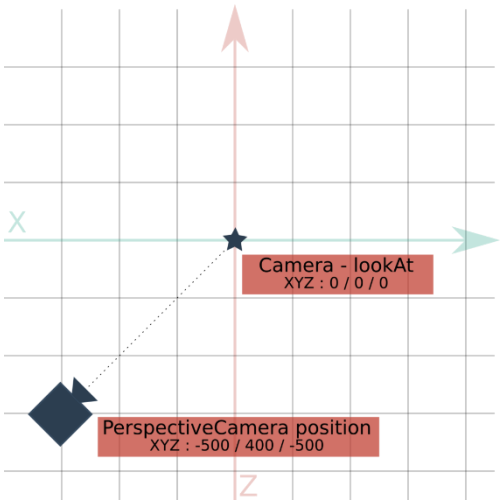
Vous utiliserez ici la projection perspective grâce à `PerspectiveCamera`. Cette dernière est conçue pour recréer le mode la projection basée sur la perspective de la vision humaine.

Avant de décrire l'ensemble de ces dernières ainsi que leurs paramètres, il faut prendre un peu recul et se poser quelques questions :

Vous allez placer une caméra dans un environnement en 3 dimensions …

- Où va se trouver la scène dans cet environnement que la caméra doit observer
- Où positionner la caméra en conséquence

Le schéma ci-contre propose une représentation dans un repère spatial de la position de la caméra et son point d'observation.



Maintenant que ces questions sont posées et qu'un début de réponse est donné, il est temps de regarder les méthodes de la classe PerspectiveCamera.

Son constructeur prend 4 paramètres :

- Le FOV (Field of View ou angle de vue de l'objectif, bien connu des joueurs de FPS (mais pas que) généralement compris entre 60 et 75 (75 dans l'exemple qui suit)
- Le ratio de la vue de la caméra, couramment le rapport de largeur de la fenêtre par sa hauteur)
- La distance de vue minimale (ici vous mettrez 1)
- La distance de vue maximale (ici vous mettez une valeur assez grande 10000)

Pour ajouter cela à votre script, dans votre fonction *start* :

```
[.. votre code déjà écrit ..]
var camera;
function start()
{
[.. votre code déjà écrit ..]
// ---------- CAMERA ----------
camera = new THREE.PerspectiveCamera( 75, window.innerWidth / window.innerHeight, 1, 10000 );
camera.position.set( 150, 150, 150);
camera.lookAt(new THREE.Vector3(0,0,0));
scene.add( camera );
}
```

Trois autres nouvelles méthodes viennent d'apparaître, ainsi qu'une propriété :

- lookAt() : la méthode *lookAt()*, prend elle aussi 3 paramètres correspondants aux coordonnées du point dans l'espace (de la scène 3D) ou l'on souhaite que la caméra « regarde »,
- add() : la méthode *add()* qui appartient ici à l'objet *scene,* permet d'ajouter un objet à la scène 3D, en l'occurrence ici, votre objet *camera*. Cette méthode sera utilisée à nouveau plus tard pour ajouter d'autres objets à la scène.

Dans Three.js, chaque objet de l'univers propose une méthode add .

Cette méthode permet d'établir une relation parent/enfant entre deux éléments. Ainsi, pour ajouter la caméra à la liste des enfants de notre scène, nous utiliserons cette fonctionnalité

- position.set() : la propriété position permet d'obtenir et manipuler (grâce à la méthode *set* entre autres) la position dans l'espace de la caméra en fonction des axes (X,Y et Z).

D'autre méthodes et propriétés sont documentées sur le site dont le lien est en introduction de ce TD / TP.

Éclairage direct et ambiant d'une scène

Vous allez bien ajouter un premier objet à votre scène, mais avant cela, il faut pouvoir voir cet objet.

Si vous disposez dans une pièce fermée (votre scène) un objet et une caméra pour filmer cet objet (perspectiveCamera ici), vous ne verrez rien sur les enregistrements, car sans éclairage, les objets filmés ne seront pas visibles.

Il faut donc ajouter à cette scène une source de lumière.

Si vous allumez une ampoule dans une pièce sombre, les objets de cette pièce recevront la lumière de deux manières :

- Éclairage direct : rayons lumineux qui proviennent directement de l'ampoule et éclaire l'objet.
- Éclairage indirect : rayons lumineux qui ont rebondi sur les murs et autres objets de la pièce avant de heurter un objet, changeant de couleur et perdant en intensité à chaque rebond.

En correspondance avec celles-ci, les classes légères de three.js sont divisées en deux types :

- Lumières directes, qui simulent un éclairage direct.
- Lumières d'ambiance, qui constituent un moyen peu coûteux et quelque peu crédible de simuler un éclairage indirect.

Vous allez d'abord ajouter une source de lumière ambiante.

L'éclairage globale indirect : AmbientLight

Pour créer et ajouter une source de lumière ambiante (ou indirect), comme son nom peut le laisser présager, il n'est pas nécessaire d'ajouter une position dans l'espace.

Commencez par instancier un objet de la classe AmbientLight puis ajoutez le à la scène comme vous aviez dû le faire pour la *camera*.

Son constructeur prend 2 paramètres :

- Couleur de l'éclairage – Facultatif – 0xFFFFFF par défaut
- Intensité de l'éclairage – Facultatif – 1 par défaut

```
[.. votre code déjà écrit ..]
function start()
{
[.. votre code déjà écrit ..]
// ---------- LIGHTS ----------
    var myAmbientLight = new THREE.AmbientLight( 0xd0d0d0, 0.3 );
    scene.add( myAmbientLight );
}
```

L'éclairage directionnel : DirectionalLight

Il est courant d'avoir besoin d'une source de lumière plus directe sur un ou plusieurs objets spécifiques et de donner un éclairage naturel (AmbientLight éclaire globalement les objets dans toutes les directions de l'ensemble de leurs faces, c'est pour cela que l'on donne une valeur volontairement basse pour son intensité).

Il existe pour cela un autre type de source lumineuse dont le nom de la classe est DirectionalLight qui simule la lumière du soleil ou d'une autre source lointaine très brillante. Il existe au total quatre types de lumière directe disponibles dans le noyau three.js, chacun simulant une source de lumière commune du monde réel:

- DirectionalLight=> Lumière du soleil
- PointLight=> Ampoules
- RectAreaLight=> Éclairage en bandes ou fenêtres lumineuses
- SpotLight=> Projecteurs

Par défaut la source de lumière pointe vers le centre du repère (0,0,0). Sa position par défaut est au-dessus (0,1,0).

Il est possible de modifier ces valeurs grâce à plusieurs propriétés (target() qui attend l'objet à éclairer, et position() qui attend un objet Vector3D pour la position de départ).

Créez une source de lumière directionnelle avec les valeurs par défaut, avec le constructeur qui attend les mêmes paramètres que ambientLight :

```
[.. votre code déjà écrit ..]
function start()
{
[.. votre code déjà écrit ..]
// ---------- LIGHTS ----------
        var directionalLight = new THREE.DirectionalLight( 0xffffff, 0.6 );
        scene.add( directionalLight );
}
```

Un Cube en 3D

Maintenant que la scène est en place, il ne reste plus qu'à y placer au moins un premier objet.

Mais comment créer un objet avec ThreeJS.

Vous aviez vu plus tôt qu'un objet au sens « physique » du terme (ici, ce sera un cube), est géré par la classe Mesh (cf. schéma du début).

Vous aviez vu également qu'un Mesh était composé de deux autres objets : *Geometry* et *Materiel* qui définissent respectivement sa forme et sa matière / texture (et les propriétés de réflexion notamment qui y sont liées).

Pour définir d'abord sa forme, et après consultation de la documentation, on peut voir qu'il existe plusieurs sous-classes de *Geometry.*

Pour produire un cube, la plus adaptée dans notre cas est BoxGeometry, dont le constructeur prend trois paramètres correspondants à la largeur (X), la hauteur (Y) et la profondeur (Z).

Si l'on s'intéresse maintenant au *Material* de notre cube, on découvre également dans la documentation qu'il en existe de nombreux avec des propriétés différentes.

Vous allez, pour l'exemple, créer un premier cube dont la matière aura une propriété de réflexion de la lumière (comme du métal).

Celui qui semble se prêter le plus à l'exercice est *MeshStandardMaterial*. (Mais vous pouvez en tester d'autres) .

Le constructeur de *Mesh* prend ensuite 2 paramètres, un objet *Geometry* (ici un *BoxGeometry*) et un *Material* (ici un *MeshStandardMaterial*).

Ci-dessous une proposition de code, après quelques essais de couleur, et de valeurs pour la réflexion. (Essayez plusieurs valeurs pour trouver celles qui conviennent le plus).

```
[.. votre code déjà écrit ..]
var cube;
function start()
{
[.. votre code déjà écrit ..]
// ---------- OBJECTS ----------
        const geometry = new THREE.BoxGeometry( 15, 15, 15 );
        const material = new THREE.MeshStandardMaterial( {color: 0x00ffff, metalness: 0.70, roughness:
0.40, envMaps: 'reflection'} );
        cube = new THREE.Mesh( geometry, material );
        scene.add( cube );
}
```

Partie 3 : Rendu 3D de la scène et animation simple

Voilà l'essentiel est fait, et la scène est prête. Il ne reste plus qu'à lancer le rendu de la scène pour la voir apparaitre.

Pour rendre la scène, il faut utiliser, comme vous l'aviez vu au tout début du TD, une des classes héritées de la classe *Renderer.*

Vous aviez pu voir qu'il en existe pour un rendu différé, pour un rendu en cache, et un, que vous allez utiliser maintenant pour un rendu direct. Il s'agit de WebGLRenderer.

Pour cela, utilisez votre objet de type `WebGLRenderer` instancié dans la fonction `start`. Cette instance met à disposition une méthode de classe `render`, qui prend deux paramètres : la *scene* à calculer, et la *camera* observant la scène.

Cela donne le code suivant à ajouter pour votre scène 3D.

```
[...]
function start()
{
[...]
// ---------------- RENDER SCENE ----------------
        renderer.render( scene, camera );
}
```

Voilà, votre « magnifique » cube doit être affiché à l'écran.

Vous pourriez vous arrêter là, mais vous allez ajouter un peu d'animation à ce cube.

En effet, maintenant que le cube est affiché, il est possible de lui appliquer une transformation, pour l'exercice, une rotation.

Comment faire effectuer une rotation permanente (animée donc) à votre cube.

Si vous appliquez une rotation autour de son axe à son point d'origine, le cube va effectivement tourner, mais vous ne verrez pas le cube tourner, vous verrez juste le cube rendu à l'écran dans une position légèrement différente.

Pour voir une rotation à l'écran, il faut appliquer une rotation au cube, à chaque rendu de la scène.

Pour cela vous devez créer une fonction *toRender* qui sera utilisée comme boucle principale de rendu, en remplacement de votre appel de méthode pour le rendu précédent (à la place donc de *renderer.render(scene, camera)* ajoutée dans la fonction *start* précédemment.

Création de la boucle principale

Créez une fonction *toRender* dans votre script (en dehors de la fonction *start*), cette fonction sera chargée d'effectuer des actions de rendu de la scène (avec rotation ou non).

```
function toRender()
{
        renderer.render( scene, camera ) ;
}
```

Remplacez maintenant l'appel de la méthode *renderer.render()* par un appel à votre fonction *toRender()* dans la fonction *start()*.

```
[...]
function start()
{
[...]
// ---------- RENDER SCENE ----------
        toRender();
}
```

Mise à jour du rendu 3D et application de la rotation

Vous allez faire tourner votre cube sur deux axes (X et Y).

Pour rappel, chaque objet de la scène possède une valeur de rotation pour chaque axe X, Y et Z.

Vous allez donc à chaque tour de boucle, appliquer une rotation (avec rotateX et rotateY) sur l'axe X et l'axe Y, pour faire entrer le cube en rotation.

```
function toRender()
{
        // fait tourner le cube à chaque appel de la fonction
        cube.rotateX(0.006);
        cube.rotateY(0.02);
        renderer.render( scene, camera );
}
```

Reste maintenant à faire que votre fonction toRender soit rappelée automatiquement afin que la scène soit recalculée et réaffichée à intervalle régulier avec le cube dans une position mise à jour.

Pour cela, vous allez vous appuyer sur une fonction très particulière de javascript (et non liée à ThreeJS) appelée *requestAnimationFrame*.

requestAnimationFrame est une méthode de l'objet *window* JavaScript adaptée à ce besoin, elle indique au navigateur qu'on souhaite exécuter une animation en exécutant une fonction spécifique passée en paramètre avant le prochain rafraîchissement à l'écran du navigateur (lié au taux de rafraichissement de l'écran, sur la plupart, 60 fois par seconde).
(Réf : https://developer.mozilla.org/fr/docs/Web/API/window/requestAnimationFrame)

Si vous souhaitez animer une nouvelle frame durant le prochain affichage, la fonction de rappel doit de nouveau appeler la méthode requestAnimationFrame(). Autrement dit, requestAnimationFrame() ne fonctionne qu'une fois.

Il faut donc, dans la fonction toRender, appeler cette fonction *requestAnimationFrame*, avec comme paramètre la fonction toRender elle-même, ce qui donne :

```
function toRender()
{
        // fait tourner le cube à chaque appel de la fonction
        cube.rotateX(0.006); // pivote sur son axe x
        cube.rotateY(0.02); // pivote sur son axe y
        renderer.render( scene, camera );
        requestAnimationFrame(toRender);
}
```

Partie 4 : Création d'objet complexe par assemblage (partie 1)

Vous avez créé ici un premier élément simple, un cube, mais d'autres objets simples sont disponibles : torus, cône, cylindre, sphère et quelques autres (cf documentation), pour lesquels ThreeJS fourni des objets prédéfinis.

Il est possible d'assembler ces objets de plusieurs façons :

- Par association de type filiation
- Par groupage

Par filiation

Pour associer des objets par filiation la procédure est somme toute assez simple, il suffit d'ajouter (grâce à la méthode *add* d'un objet, un autre objet préalablement défini.

Ecrivez le code nécessaire à l'ajout d'une sphère (en vous inspirant de celui utilisé pour créer un cube), puis grâce à la méthode *add* du cube, ajoutez la sphère en temps qu'enfant du cube.

```
// création du mesh et ajout comme fils du premier mesh
var SphereFille = new THREE.Mesh(new THREE.SphereGeometry(7.5 , 32 , 16));
cube.add(SphereFille);
SphereFille.position.set(0,15,0);
```

Attention : le positionnement d'un mesh enfant se fait relativement à son parent.

Déplacez maintenant le cube en modifiant sa position à gauche ou à droite à l'aide de la méthode *set* de sa propriété *position*.

```
cube.position.set = (-15,0,0); // on déplace le cube à gauche par exemple
```

Le cube se déplace avec ses meshs enfants (la sphère pour cet exemple), même si vous n'avez pas changé sa position directement.

Par groupage

Une scène peut contenir plusieurs groupes et objets indépendants.

De plus, un groupe n'a pas de représentation visuelle, mais cela reste un objet à proprement parlé.

On peut donc lui appliquer des transformations et les animer, pour par exemple donner l'impression d'un objet tournant autour d'un autre.

Pour associer des objets par groupage il suffit de créer un groupe puis d'y ajouter les éléments qui le constitueront grâce à sa méthode *add*, faisant de ces objets les enfants du groupe. (Comme dans la filiation).

Partie 5 : Création d'objet complexe from scratch

Vous avez créé ici un premier élément simple, un cube, d'autres objets simples sont disponibles : torus, cône, cylindre, et quelques autres (cf. documentation), pour lesquels ThreeJS fournit des objets prédéfinis.

Il est cependant possible de créer des objets sur mesure, composés d'autant de polygones (des triangles qui seront appelés *faces* dans la suite du cours).

Ces polygones (ou faces) sont constitués de sommets appelés vertices (au singulier : vertex), qui sont définis par leurs coordonnées dans l'espace.

Pour générer un objet de la sorte avec ThreeJS, il faut définir un objet Geometry (donc non prédéfini), et spécifier ensuite l'ensemble des vertices le constituant, puis préciser les *faces* qui le composent.

Vous allez créer une forme simple et familière (un cube), sur mesure, en définissant l'ensemble des vertices qui le constitue, ainsi que les faces (polygone triangulaire) qui composent également.

Pour produire un tel objet, il faut déclarer un objet de type geometry générique, un BufferGeometry.

```
const geometry = new THREE.BufferGeometry();
```

Une fois cet objet créé, il faut lui fournir un tableau des vertices (sommets des triangles) composant la forme attendue (augmenter la valeur de 50 si rien n'est visible) :

```
const vertices = new Float32Array( [
        -50.0, -50.0,  50.0, // vertice 0
         50.0, -50.0,  50.0, // vertice 1
         50.0,  50.0,  50.0, // vertice 2
        -50.0,  50.0,  50.0, // vertice 3
        50.0, -50.0, -50.0, // vertice 4
        50.0,  50.0, -50.0, // vertice 5
        -50.0, -50.0, -50.0, // vertice 6
        -50.0,  50.0, -50.0, // vertice 7
] );
```

Puis définir chacune des faces en assemblant les vertices ensembles :

```
const indices = [
        0, 1, 2,
        2, 3, 0,
        1, 4, 5,
        5, 2, 1,
        4, 6, 7,
        7, 5, 4,
        6, 0, 3,
        3, 7, 6,
        3, 2, 5,
        5, 7, 3,
];
```

Enfin, vous devez fournir à l'objet geometry le tableau de vertices, et le tableau des faces pour créer l'objet attendu.

```
geometry.setIndex( indices );
geometry.setAttribute( 'position', new THREE.BufferAttribute( vertices, 3 ) );
```

Pour finir, créez un Mesh à l'aide de ce Geometry, ainsi qu'un Material, puis ajoutez ce mesh à la scène.
Un cube est désormais visible, constitué à base de polygones définis manuellement.